

HPC Lab Final Report: 鲲鹏 920F TRSM

- 1 实验目标 2
- 2 问题定义与实验环境 2
 - 2.1 TRSM 计算结构 2
 - 2.2 实验环境与测试规模 2
- 3 Baseline 与优化思路 2
- 4 最终实现 3
 - 4.1 按形状选择递归或块算法 3
 - 4.2 对角块求解与向量化 3
 - 4.3 尾部更新与线程方向 4
 - 4.4 叶子块求解的 W-technique (V16) 4
 - 4.5 OpenMP 与 KBLAS 的组合 4
- 5 正确性与性能结果 5
- 6 尝试、取舍与局限 5
 - 6.1 SME 候选路径的取舍 5
 - 6.2 结构变体的系统否决 5
- 7 总结 6

1 实验目标

给定下三角矩阵 L 和右端矩阵 B ，程序求解

$$LX = B, \quad L \in \mathbb{R}^{m \times m}, \quad X, B \in \mathbb{R}^{m \times n} \quad (1)$$

2 问题定义与实验环境

2.1 TRSM 计算结构

本题的 L 为下三角矩阵，求解过程存在沿行方向的前向依赖。将 L 按行块划分后，当前对角块可以先完成三角求解，再使用已经求出的上方右端块更新剩余部分：

```
for each diagonal block i:
    solve L[i:i+bs, i:i+bs] * X[i:i+bs, :] = B[i:i+bs, :]
    B[i+bs:, :] -= L[i+bs:, i:i+bs] * X[i:i+bs, :]
```

其中对角块求解是串行依赖最强的部分，尾部更新则是矩阵乘法，适合调用 KBLAS 的 DGEMM。该分解把一个整体的三角求解转化为“对角小块求解加尾部矩阵更新”的重复过程，也为根据矩阵形状选择并行方向提供了接口。

2.2 实验环境与测试规模

实验记录对应深超算鲲鹏集群的计算节点，架构为 AArch64，处理器为鲲鹏 920F。程序使用 GCC、OpenMP、numactl 和 KBLAS，运行时固定为 38 个 OpenMP 线程，并使用 numactl -N 1 将计算限制在一个 NUMA 节点。环境脚本将 KBLAS 头文件、动态库和 BiSheng HPCKit 的 OpenMP 运行库加入搜索路径。

用例	矩阵规模	形状特征	最终路径
Case 1	$m = 512, n = 19968$	小 m 、宽 n	递归, leaf = 48, N-split
Case 2	$m = 2432, n = 17024$	中等 m 、宽 n	递归, leaf = 32, N-split
Case 3	$m = 17024, n = 512$	大 m 、窄 n	NB = 224, M-split

Table 1: TRSM 三组正式测试规模与路径选择

3 Baseline 与优化思路

TRSM 的性能不能只由一个固定分块参数决定。三个测试用例的 m 和 n 差异较大：Case 1 和 Case 2 的右端矩阵很宽，尾部更新按 n 分片可以让线程获得足够的连续列工作；Case 3 的右端矩阵较窄，如果继续沿 n 方向切分，线程并行度会受限，因此改为沿剩余行按 m 分片。最终实现将这一判断固化为按形状分支的 V9 路径，并在此基础上叠加两项改进：宽 n 更新的二维切分 (V14) 与叶子块求解的 W-technique (V16)。

优化过程中保留了三个原则：第一，矩阵运算的主体交给成熟的 KBLAS DGEMM；第二，KBLAS 在每次调用中固定为单线程，避免其内部线程与外层 OpenMP 嵌套造成过量线程和资源争用；第三，只在足够大的更新上建立 OpenMP 并行区，小矩阵更新直接串行调用 KBLAS，减少线程创建与同步开销。

4 最终实现

4.1 按形状选择递归或块算法

正式 trsm.c 的入口是 l_trsm。当 m 不超过 1024 时，程序使用 leaf 为 48 的递归算法；当 m 大于 1024 且不超过 4096 时，使用 leaf 为 32 的递归算法；更大的 m 使用固定块大小 224 的 right-looking 块算法。递归分割点按 32 行对齐，使子问题和 OpenMP 分片保持规则的行边界。

```
if m <= 1024:
    recursive TRSM, leaf = 48
else if m <= 4096:
    recursive TRSM, leaf = 32
else:
    blocked TRSM, NB = 224
```

递归路径先求解左上子问题，再调用 dgemm_update 更新右下区域，最后求解右下子问题。大规模块路径沿对角块从上到下推进，在每个块之后更新剩余行。两条路径都保持同一个数学递推，因此只改变了任务组织方式，没有改变结果语义。

版本级记录显示，V9、V14、V16 的综合吞吐量分别为 1136.8、1154.1、1182.1 GFLOPS。V14 相对 V9 为约 1.015 倍 (+1.5%)，V16 相对 V9 为约 1.040 倍 (+4.0%)；V16 与同批 V14 (1123 GFLOPS) 交错对照时为约 1.051 倍 (+5.1%)。不同节点和不同批次的绝对吞吐量仅用于版本演进参考。

版本	主要路径	综合吞吐量	相对加速
V9	形状感知的递归或块算法	1136.8 GFLOPS	1.000 倍，基线
V14	宽 n 更新二维切分	1154.1 GFLOPS	相对 V9 为 1.015 倍，+1.5%
V16	V14 加 W-technique	1182.1 GFLOPS	相对同批 V14 为 1.051 倍，+5.1%

Table 2: TRSM 版本演进的综合加速效果

4.2 对角块求解与向量化

对角块求解按右端矩阵的列方向划分任务。对于 n 不大的用例，每个任务处理 16 列；对于宽矩阵，每个任务处理 32 列。每一行的前向代入先把当前列块读入局部累加数组，再沿已经求解的行进行乘减，最后乘以对角元素的倒数。内层列循环使用 OpenMP simd 指令，使编译器能够将连续列上的乘减和缩放映射到 ARM 向量指令。

这种安排保留了行之间不可消除的三角依赖，同时将不同右端列之间的独立性暴露给 OpenMP。相邻两行同时计算的 ILP 候选在节点实测中对 Case 3 产生 7.0% 的负收益，因此正式版本保留单行依赖路径。

W-technique 的 leaf = 64 变体在 Case 2 中使吞吐量从 1433 降至 1032 GFLOPS，约下降 28.0%；因此正式版本采用 leaf = 32 或 48。使用 dgemm 加临时缓冲的 W-technique 也没有形成增益，记录显示缓冲往返流量抵消了计算收益。

4.3 尾部更新与线程方向

尾部更新的数学形式为

$$B_2 \leftarrow B_2 - L_{21}B_1 \quad (2)$$

实现根据更新规模在两种并行方向之间选择：当 n 大于 4096 时，沿列方向分片，每个线程处理连续的 n 列；否则沿行方向分片，每个线程处理连续的剩余行。每个线程调用单线程 KBLAS DGEMM，并写入互不重叠的 B2 区域。这样既能复用 KBLAS 的高效矩阵乘内核，又能避免多个外层线程同时写同一输出元素。

在纯列分片下，38 个线程都要完整读取一遍乘数面板 L_{21} ，该面板的重复读取达到 38 份。V14 将宽 n 更新改为二维切分：38 个线程划分为 2 个行组乘 19 个列组，行组内的 19 个线程共享同一份行区间的 L_{21} ，面板重复读取降到 19 份；代价是 B 面板的重复读取翻倍，但每个线程获得的列条带宽度也翻倍，流式访问效率更高。节点探针显示，Case 2 顶层更新形状在此切分下从约 1684 GFLOPS 提升到约 1835 GFLOPS。该路径由 TRSM_WIDE_2D 宏控制，仅在 n 大于 4096 且更新行数不低于 256 时启用，更深的递归小更新仍走纯列分片，Case 3 的行分片不受影响。

其他更新组织方式均未超过该路径：Case 3 的 k 聚合使性能下降 2.5%，宽 n 用例的扁平分块替代递归下降 35% 到 40%，持久区域加固定列条带下降 60%，宽 n 强制 M-split 下降 66%。扁平 NB = 608 重构没有可测增益，left-looking 更新族仅达到 1399 到 1672 GFLOPS，低于右视路径记录的 1884 GFLOPS。

4.4 叶子块求解的 W-technique (V16)

原实现对角叶子块采用标量前代消元：行 i 依赖前 $i-1$ 行的解，形成严格的串行链，聚合速率只有约 110 GFLOPS，在 Case 2 中占据 12.1 ms（占总时间 15%）。V16 将其替换为选择性求逆：先以 $O\left(\frac{n_b^3}{6}\right)$ 的串行成本求出 32 到 48 行对角块的逆矩阵（转置存储，使内层乘减为连续访存流，可被向量化），再用原地 cblas_dtrmm 按列分片计算 $B_j := L_{ji}^{-1}B_j$ 。串行链被完全消除，求解速率提升到实测的 480 到 860 tri-GFLOPS（形状相关）。64 行以上的大对角块（Case 3 的 224 行块）因求逆串行成本不划算而保持原标量路径，由宏守卫限制。该替换带来的数值误差从 6.66×10^{-16} 变为 2.78×10^{-16} 到 4.44×10^{-16} 量级，仍低于阈值四个数量级。

当 rows、n、bs 三者乘积较小时，代码不建立 OpenMP 并行区而直接调用 KBLAS。这一阈值策略用于抑制递归尾部和最后一个小尾块的 fork/join 成本，尤其适合 Case 1 的小对角块和 Case 3 的末尾块。

4.5 OpenMP 与 KBLAS 的组合

KBLAS 通过构造函数调用 BlasSetNumThreads(1)，把内部 DGEMM 固定为单线程。大更新由外层 OpenMP 负责分片，形成以下层次：

层次	组件	作用
线程配置	OMP_NUM_THREADS=38	使用一个 NUMA 节点上的固定线程数
任务分片	外层 OpenMP	按列或按行切分独立的 DGEMM 子任务
数值内核	单线程 KBLAS	执行每个子矩阵乘，避免嵌套并行
数据放置	numactl -N 1	限制内存和计算的 NUMA 范围

Table 3: 最终版本的并行层次

该组合方式本身没有单独记录可复现的端到端加速数字，但线程组织的取舍有对照结果：Case 3 尝试将求解与更新流水线重叠时，最佳情况仍下降约 2%，原因是求解线程组导致更新线程组饥饿。因此正式版本保持外层 OpenMP 分片、内层单线程 KBLAS 的层次，不额外引入流水线并发。

5 正确性与性能结果

用例	规模 $m \times n$	平均时间 / ms	GFLOPS	最大绝对误差	校验
Case 1	512×19968	6.02	869.7710	$2.78e-16$	PASS
Case 2	2432×17024	71.08	1416.5514	$4.44e-16$	PASS
Case 3	17024×512	138.03	1075.0646	$1.83e-15$	PASS

Table 4: TRSM V16 正式结果记录（节点 cn22994, test_runs=5）

三组用例的显示时间总和为 215.13 ms，按 bench 使用的 FLOPs 计量，三个用例合计约 254.311137280 GFLOP，综合吞吐量约为 1182.1 GFLOPS。

指标	结果	结论
正确性	3/3 PASS, 最大误差 1.83×10^{-15}	满足 10^{-12} 阈值
平均时间总和	215.13 ms	三组正式用例合计耗时
综合吞吐量	1182.1 GFLOPS	按 bench 的 m^2n 计量

Table 5: 正式结果摘要

6 尝试、取舍与局限

6.1 SME 候选路径的取舍

实验记录中对普通 SVE、SME FMOPA 以及自有更新内核做过能力和微内核验证。节点具有 SVE2、SME 和 smef64f64 特征，实测 SVE 向量长度为 512 bit；但 HWCAP2_SVEF64MM 为 0，不能把 FMMLA 作为可靠的通用路径。KBLAS 的 DGEMM 反汇编包含 SME 的 FMOPA 指令，说明现有库已经能够使用 SME F64F64 路径。

自有 16×32 SME 更新内核在真实 load/store 条件下约为 1737 GFLOPS，加入共享 B 打包和每线程 A 打包后约为 1525 GFLOPS。分段计时修正后的 KBLAS 更新段真实速率约为 1500 GFLOPS，自有 SME 内核相对该速率没有实质优势，还需要额外的打包、尾块和线程协作逻辑，最终没有把 SME 候选接入正式 run.sh。这是一个基于端到端风险和实测结果的取舍，提交版本优先保留可复现、可验证的 KBLAS 路径。

6.2 结构变体的系统否决

在 V16 之前，还系统测过多个结构性变体，全部通过正确性检查但在同批交错对照中负收益：k 聚合（把 Case 3 的四步更新合并为 k=896 的一次大更新，破坏 B 面板的 L2 驻留，Case 3 慢 2.5%）；扁平分块替代递归（宽 N 用例的更新 k 变小，慢 35% 到 40%）；持久区域加固定列条带（同样受制于小 k 更新，慢 60%）；强制 M-split 用于宽 N（每线程行数太瘦，慢 66%）；对角求解双行 ILP（Case 3 慢 7.0%）；Case 3 的求解与更新流水线重叠（求解组占用线程造成的更新组饥饿大于重叠收益，最好情形仍慢 2%）；W-technique 经 dgemm 加临时缓冲（缓冲往返流量吃掉全部计算收益）；W-technique 下 leaf=64（Case 2 从 1433 降至 1032

GFLOPS); 扁平 NB=608 重构 (探针显示与树结构同速率族, 无增益); left-looking 更新族 (1399 到 1672 GF, 全面劣于右视的 1884)。这些负结果与 V14、V16 的正收益一起, 说明当前算法结构已处于该引擎下的较优位置。

7 总结

本实验针对下三角求解的形状差异, 构建了递归和块算法结合的混合 TRSM 实现。小、中规模使用不同 leaf 的递归路径, 大规模使用 224 行块并沿 M 方向分片; 宽右端矩阵的更新在 V14 中升级为 2 行组乘 19 列组的二维切分, 把乘数面板的重复读取从 38 份降到 19 份; V16 进一步将对角叶子块的串行前代消元替换为选择性求逆加原地 dtrmm, 消除了求解段的行串行链。尾部更新交给单线程 KBLAS DGEMM, 并由外层 OpenMP 提供任务级并行, 从而避免嵌套 OpenMP 的线程争用。

最终三组测试全部通过 10^{-12} 正确性阈值, 最大误差为 1.83×10^{-15} , 综合吞吐量为 1182.1 GFLOPS, 相对 V9 基线提升约 4%, 相对 V14 提升 5.1%。分段下限的求和分析进一步表明, 该结果已接近 KBLAS 单线程 DGEMM 引擎在此节点上的可达上限; 继续逼近更高目标需要引入非 KBLAS 的更新内核, 而这已被端到端实测证明收益有限。